AI-ASSISTED CODING

ASSIGNMENT 2.5

NAME:V.SIDDARTHA
ROL:2303A52062
BATCH:45

#TASK 1

Task 1: Refactoring Odd/Even Logic (List Version)

❖ Scenario:

You are improving legacy code.

❖ Task:

Write a program to calculate the sum of odd and even numbers in a list,
then refactor it using AI.

❖ Expected Output:

❖ Original and improved code

#PROMPT

Write a Python program to find the sum of odd and even numbers in a list using basic loops.
Improve the Python code to calculate sum of odd and even numbers using list comprehension or built-in functions.

#CODE

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
sum_odd = 0
sum_even = 0
for num in numbers:
    if num % 2 == 0:
        sum_even += num
    else:
        sum_odd += num
print("Sum of odd numbers:", sum_odd)
print("Sum of even numbers:", sum_even)
# Using list comprehension
sum_odd = sum(num for num in numbers if num % 2 != 0)
sum_even = sum(num for num in numbers if num % 2 == 0)
print("Sum of odd numbers (list comprehension):", sum_odd)
print("Sum of even numbers (list comprehension):", sum_even)
```

#OUTPUT

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/__pycache__/assign_12.3/ASSIGNMENT 2.5.PY"
Sum of odd numbers: 25
Sum of even numbers: 30
Sum of odd numbers (list comprehension): 25
Sum of even numbers (list comprehension): 30
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>

powershell
Python
Python
```

#TASK 2

Task 2: Area Calculation Explanation

❖ Scenario:

You are onboarding a junior developer.

❖ Task:

Ask Gemini to explain a function that calculates the area of different shapes.

❖ Expected Output:

➢ Code

➢ Explanation

#PROMPT

Function to calculate area of different shapes. Write a Python function to calculate the area of shapes like circle, rectangle, and triangle, and explain the code in simple words.

#CODE

```
import math
def calculate_area(shape, *dimensions):
    if shape == "circle":
        radius = dimensions[0]
        return math.pi * radius ** 2
    elif shape == "rectangle":
        length, width = dimensions
        return length * width
    elif shape == "triangle":
        base, height = dimensions
        return 0.5 * base * height
    else:
        raise ValueError("Unsupported shape")
```

```
# Example usage:
circle_area = calculate_area("circle", 5)
rectangle_area = calculate_area("rectangle", 4, 6)
triangle_area = calculate_area("triangle", 4, 5)
print("Area of circle:", circle_area)
print("Area of rectangle:", rectangle_area)
print("Area of triangle:", triangle_area)
```

#OUTPUT

```
Sum of odd numbers (list comprehension): 25
Sum of even numbers (list comprehension): 30
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppDat
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI -
isted coding/__pycache__/assign_12.3/ASSIGNMENT 2.5.PY"
Area of circle: 78.53981633974483
Area of rectangle: 24
Area of triangle: 10.0
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

#JUSTIFICATION

This code defines a function called calculate_area that takes a shape type and its dimensions as input. Depending on the shape specified, it calculates the area using the appropriate formula. For a circle, it uses the formula $A = \pi r^2$, for a rectangle it uses A = length × width, and for a triangle it uses A = 0.5 × base × height. The function raises an error if an unsupported shape is provided.

#TASK3

Task 3: Prompt Sensitivity Experiment

❖ Scenario:

You are testing how AI responds to different prompts.

❖ Task:

Use Cursor AI with different prompts for the same problem and observe code changes.

❖ Expected Output:

➢ Prompt list

➢ Code variations

#PROMPT

Find factorial of a number. Write a Python program to find factorial of a number using a loop. Write a Python program to find factorial of a number using recursion. Write an efficient Python program to find factorial using built-in functions.

#CODE

def factorial_loop(n):

```
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
# Using recursion
def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive(n - 1)
# Using built-in functions
import math
def factorial_builtin(n):
    return math.factorial(n)
# Example usage:
number = 5
print("Factorial using loop:", factorial_loop(number))
print("Factorial using recursion:", factorial_recursive(number))
print("Factorial using built-in function:", factorial_builtin(number))
```

#OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/__pycache__/assign_12.3/ASSIGNMENT 2.5.PY"
Factorial using loop: 120
Factorial using recursion: 120
Factorial using built-in function: 120
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

#JUSTIFICATION

This code defines three functions to calculate the factorial of a number. The first function, factorial_loop, uses a for loop to multiply all integers from 1 to n. The second function, factorial_recursive, uses recursion by calling itself with decremented values of n until it reaches the base case (0 or 1). The third function, factorial_builtin, utilizes the built-in math.factorial function for an efficient calculation.